

Explicação dos códigos de verificação SystemVerilog

Os dois arquivos .sv possuem funções complementares. O `mac_rx_mockup.sv` representa o circuito sob teste e o `tb_mac_mockup.sv` controla a simulação, aplica os estímulos e verifica os resultados. Essa separação permite substituir posteriormente o mockup pelo Packet Parser real sem alterar a estrutura principal da testbench.

1 Visao geral do fluxo

O `stream_words.txt` contém as palavras do quadro Ethernet divididas em blocos de 64 bits. O `testbench` lê essas palavras e aplica os sinais ao `mac_rx_mockup.sv`. O DUT mockup conta os bytes, verifica os controles e captura o timestamp do primeiro beat de cada pacote. Ao final, o testbench gera `resultado_simulacao.txt`.

2 Interface usada

A interface RX não possui handshake com TREADY. Por isso, o testbench envia uma palavra válida por ciclo durante o quadro.

O fim do pacote é indicado por TLAST.

O início é identificado pelo primeiro ciclo com TVALID igual a 1 depois de um período ocioso ou depois do TLAST anterior.

Sinal	Largura	Função
<code>rx_axis_aresetn</code>	1 bit	Reset ativo em nível baixo.
<code>rx_axis_tdata</code>	64 bits	Dados do quadro Ethernet; 8 bytes por ciclo.
<code>rx_axis_tkeep</code>	8 bits	Indica quais bytes de tdata são válidos.
<code>rx_axis_tvalid</code>	1 bit	Indica que o beat apresentado é válido.
<code>rx_axis_tlast</code>	1 bit	Indica o último beat do pacote; corresponde ao fim de pacote.
<code>rx_axis_tuser</code>	1 bit	Indica o resultado da recepção; para quadro válido, vale 1 no beat com TLAST.
<code>tsu_timestamp_ns</code>	64 bits	Entrada auxiliar do mockup com o timestamp do TSU em nanossegundos.

Portanto, não tem uma coluna SOP e EOP: o TLAST exerce essa função de encerramento do pacote.

3 Arquivo `mac_rx_mockup.sv`

O arquivo `mac_rx_mockup.sv` contém o `mac_rx_mockup`. É um DUT usado para validar o fluxo de leitura, transmissão dos estímulos e verificação dos controles

Saída	Uso no testbench
<code>packet_done</code>	Pulso de um ciclo quando TLAST encerra o pacote.
<code>packet_error</code>	Indica TKEEP inválido ou TUSER inválido no encerramento.
<code>packet_id</code>	Número sequencial do pacote concluído.
<code>packet_byte_count</code>	Quantidade de bytes válidos calculada a partir de TKEEP.
<code>packet_word_count</code>	Quantidade de beats recebidos no pacote.
<code>packet_timestamp_ns</code>	Timestamp capturado no primeiro beat válido do pacote.

A função `count_valid_bytes` recebe TKEEP e conta quantos bits estão em 1. Esse valor representa a quantidade de bytes válidos no beat.

```
count_valid_bytes = count_valid_bytes + keep[index];
```

A função `valid_keep` verifica se TKEEP é iniciada no bit menos significativo. São aceitos 01, 03, 07, 0F, 1F, 3F, 7F e FF.

O bloco `always_ff` é executado na borda de subida do clock. Quando o reset está ativo, todos os contadores e resultados retornam a zero. Quando `tvalid` está ativo, o DUT conta os bytes, incrementa as palavras e armazena o timestamp do primeiro beat. Quando `tlast` chega, o pacote é finalizado e os resultados são disponibilizados para o testbench.

```
if (rx_axis_tvalid) begin
    byte_count <= byte_count + count_valid_bytes(rx_axis_tkeep);
    word_count <= word_count + 1;

    if (rx_axis_tlast) begin
        packet_done <= 1'b1;
    end
end
```

O tuser é validado no beat que possui tlast. Para um pacote válido, o último beat deve apresentar tuser igual a 1. Os beats anteriores podem apresentar tuser igual a 0.

4 Arquivo tb_mac_mockup.sv

O arquivo tb_mac_mockup.sv controla a simulação. Ele gera os sinais de ambiente, abre o arquivo de estímulos, envia cada linha para o DUT e compara os resultados observados com os valores esperados calculados pelo próprio testbench.

```
localparam string STREAM_FILE = "stream_words.txt";
localparam string RESULT_FILE = "resultado_simulacao.txt";
localparam integer MAX_PACKETS = 10000;
localparam integer CLOCK_PERIOD_NS = 10;
```

STREAM_FILE define o arquivo de entrada. RESULT_FILE define o relatório criado pela simulação. MAX_PACKETS limita a quantidade de pacotes aceitos. **CLOCK_PERIOD_NS define um clock de 10 ns, equivalente a 100 MHz.**

O clock alterna a cada metade do periodo. O reset rx_axis_aresetn é ativo em nível baixo. Durante o reset, o timestamp tsu_timestamp_ns permanece em zero. Após a liberacao do reset, o contador aumenta 10 ns a cada posedge.

```
always #(CLOCK_PERIOD_NS / 2) rx_clk = ~rx_clk;

always_ff @(posedge rx_clk or negedge rx_axis_aresetn) begin
    if (!rx_axis_aresetn)
        tsu_timestamp_ns <= 64'd0;
    else
        tsu_timestamp_ns <= tsu_timestamp_ns + CLOCK_PERIOD_NS;
end
```

O testbench armazena o timestamp do primeiro beat de cada pacote em expected_first_timestamp. O DUT produz o mesmo valor em packet_timestamp_ns.

A função *\$fopen* abre stream_words.txt. *\$fgets* lê uma linha de cada vez e *\$sscanf* separa os campos. Linhas de cabeçalho ou comentário são ignoradas

```
stream_fd = $fopen(STREAM_FILE, "r");

while ($fgets(line, stream_fd) != 0) begin
    rc = $sscanf(line, "%h %h %d %d %d",
        data_i, keep_i, valid_i, last_i, user_i);
end
```

Quando a linha possui quatro campos, o testbench infere `user_i = last_i`. Assim, o arquivo pode ser gerado apenas com `tdata`, `tkeep`, `tvalid` e `tlast`. Quando houver cinco campos, o quinto campo será usado como `tuser`.

Depois da leitura, os valores são colocados nas portas do DUT. O testbench aplica os sinais em uma borda de descida e o DUT os amostra na borda de subida seguinte.

```
rx_axis_tdata = data_i;  
rx_axis_tkeep = keep_i;  
rx_axis_tvalid = (valid_i != 0);  
rx_axis_tlast = (last_i != 0);  
rx_axis_tuser = (user_i != 0);  
  
@(posedge rx_clk);
```

Verificação	Como é realizada
Reset	Confere se <code>packet_done</code> e <code>packet_id</code> permanecem zerados durante o reset.
TKEEP	Confere se a máscara é uma das formas válidas e soma os bytes válidos.
TLAST	Confere se o DUT finaliza o pacote no mesmo beat que o estímulo.
TUSER	Confere <code>tuser=1</code> no beat final de um quadro válido.
Contagem	Compara bytes e palavras esperados com os resultados do DUT.
Timestamp	Compara o timestamp capturado pelo DUT com o timestamp do primeiro beat.

Geração do arquivo TXT

O testbench cria o relatório com `$fopen` e escreve os dados usando `$fdisplay`. O `$display` mostra a informação no terminal. O `$fdisplay` grava a mesma informação no arquivo `resultado_simulacao.txt`.

```
result_fd = $fopen(RESULT_FILE, "w");

$fdisplay(result_fd,
    "pacote=%0d word=%0d tsu=%0d ns",
    packet_index + 1,
    word_index,
    tsu_timestamp_ns);

$fclose(result_fd);
```

5 Arquivo stream_words.txt

O stream_words.txt é a entrada da testbench. Cada linha representa um beat AXI-Stream de 64 bits. A ordem das linhas é a ordem em que os dados devem ser apresentados ao DUT.

```
# tdata tkeep tvalid tlast
0002012000000002 FF 1 0
0045000801100000 FF 1 0
000000003B360020 3F 1 1
```

O primeiro beat do pacote não é identificado por SOP. O testbench reconhece o início quando encontra o primeiro beat válido depois de um ciclo ocioso ou do TLAST anterior.

6 Arquivo resultado_simulacao.txt

Resultado da simulacao do MAC RX mockup

Campos: pacote word tsu ns tdata tkeep tvalid tlast tuser

```
pacote=1 word=0 tsu=10 ns tdata=0002012000000002 tkeep=ff tvalid=1 tlast=0 tuser=0
pacote=1 word=1 tsu=20 ns tdata=0045000801100000 tkeep=ff tvalid=1 tlast=0 tuser=0
pacote=1 word=2 tsu=30 ns tdata=0640000001002800 tkeep=ff tvalid=1 tlast=0 tuser=0
pacote=1 word=3 tsu=40 ns tdata=a8c00a0aa8c060e5 tkeep=ff tvalid=1 tlast=0 tuser=0
pacote=1 word=4 tsu=50 ns tdata=0000500000c0140a tkeep=ff tvalid=1 tlast=0 tuser=0
pacote=1 word=5 tsu=60 ns tdata=025000000000e803 tkeep=ff tvalid=1 tlast=0 tuser=0
pacote=1 word=6 tsu=70 ns tdata=000000003b360020 tkeep=3f tvalid=1 tlast=1 tuser=1
Pacote 1 concluido: bytes=54/54 words=7/7 timestamp=10/10 ns erro=0
pacote=2 word=0 tsu=90 ns tdata=0002011000000002 tkeep=ff tvalid=1 tlast=0 tuser=0
pacote=2 word=1 tsu=100 ns tdata=0045000801200000 tkeep=ff tvalid=1 tlast=0 tuser=0
pacote=2 word=2 tsu=110 ns tdata=0640000001002800 tkeep=ff tvalid=1 tlast=0 tuser=0
pacote=2 word=3 tsu=120 ns tdata=a8c0140aa8c060e5 tkeep=ff tvalid=1 tlast=0 tuser=0
pacote=2 word=4 tsu=130 ns tdata=000000c050000a0a tkeep=ff tvalid=1 tlast=0 tuser=0
pacote=2 word=5 tsu=140 ns tdata=1250e9030000d007 tkeep=ff tvalid=1 tlast=0 tuser=0
pacote=2 word=6 tsu=150 ns tdata=000000005a2e0020 tkeep=3f tvalid=1 tlast=1 tuser=1
Pacote 2 concluido: bytes=54/54 words=7/7 timestamp=90/90 ns erro=0
pacote=3 word=0 tsu=170 ns tdata=0002012000000002 tkeep=ff tvalid=1 tlast=0 tuser=0
pacote=3 word=1 tsu=180 ns tdata=0045000801100000 tkeep=ff tvalid=1 tlast=0 tuser=0
pacote=3 word=2 tsu=190 ns tdata=0640000001002800 tkeep=ff tvalid=1 tlast=0 tuser=0
pacote=3 word=3 tsu=200 ns tdata=a8c00a0aa8c060e5 tkeep=ff tvalid=1 tlast=0 tuser=0
pacote=3 word=4 tsu=210 ns tdata=0000500000c0140a tkeep=ff tvalid=1 tlast=0 tuser=0
pacote=3 word=5 tsu=220 ns tdata=1050d1070000e903 tkeep=ff tvalid=1 tlast=0 tuser=0
pacote=3 word=6 tsu=230 ns tdata=000000005b2e0020 tkeep=3f tvalid=1 tlast=1 tuser=1
Pacote 3 concluido: bytes=54/54 words=7/7 timestamp=170/170 ns erro=0
pacote=4 word=0 tsu=250 ns tdata=0002012000000002 tkeep=ff tvalid=1 tlast=0 tuser=0
pacote=4 word=1 tsu=260 ns tdata=0045000801100000 tkeep=ff tvalid=1 tlast=0 tuser=0
pacote=4 word=2 tsu=270 ns tdata=0640000001006c00 tkeep=ff tvalid=1 tlast=0 tuser=0
pacote=4 word=3 tsu=280 ns tdata=a8c00a0aa8c01ce5 tkeep=ff tvalid=1 tlast=0 tuser=0
pacote=4 word=4 tsu=290 ns tdata=0000500000c0140a tkeep=ff tvalid=1 tlast=0 tuser=0
pacote=4 word=5 tsu=300 ns tdata=1850d1070000e903 tkeep=ff tvalid=1 tlast=0 tuser=0
pacote=4 word=6 tsu=310 ns tdata=45470000785f0020 tkeep=ff tvalid=1 tlast=0 tuser=0
pacote=4 word=7 tsu=320 ns tdata=7865646e692f2054 tkeep=ff tvalid=1 tlast=0 tuser=0
pacote=4 word=8 tsu=330 ns tdata=5448206c6d74682e tkeep=ff tvalid=1 tlast=0 tuser=0
pacote=4 word=9 tsu=340 ns tdata=0a0d312e312f5054 tkeep=ff tvalid=1 tlast=0 tuser=0
pacote=4 word=10 tsu=350 ns tdata=7865203a74736f48 tkeep=ff tvalid=1 tlast=0 tuser=0
pacote=4 word=11 tsu=360 ns tdata=6f6c2e6f6c706d65 tkeep=ff tvalid=1 tlast=0 tuser=0
pacote=4 word=12 tsu=370 ns tdata=6e6f430a0d6c6163 tkeep=ff tvalid=1 tlast=0 tuser=0
```

```

pacote=4 word=13 tsu=380 ns tdata=3a6e6f697463656e tkeep=ff tvalid=1 tlast=0 tuser=0
pacote=4 word=14 tsu=390 ns tdata=0a0d65736f6c6320 tkeep=ff tvalid=1 tlast=0 tuser=0
pacote=4 word=15 tsu=400 ns tdata=0000000000000a0d tkeep=03 tvalid=1 tlast=1 tuser=1
Pacote 4 concluido: bytes=122/122 words=16/16 timestamp=250/250 ns erro=0
pacote=5 word=0 tsu=420 ns tdata=0002011000000002 tkeep=ff tvalid=1 tlast=0 tuser=0
pacote=5 word=1 tsu=430 ns tdata=0045000801200000 tkeep=ff tvalid=1 tlast=0 tuser=0
pacote=5 word=2 tsu=440 ns tdata=0640000001006f00 tkeep=ff tvalid=1 tlast=0 tuser=0
pacote=5 word=3 tsu=450 ns tdata=a8c0140aa8c019e5 tkeep=ff tvalid=1 tlast=0 tuser=0
pacote=5 word=4 tsu=460 ns tdata=000000c050000a0a tkeep=ff tvalid=1 tlast=0 tuser=0
pacote=5 word=5 tsu=470 ns tdata=18502d040000d107 tkeep=ff tvalid=1 tlast=0 tuser=0
pacote=5 word=6 tsu=480 ns tdata=5448000021e70020 tkeep=ff tvalid=1 tlast=0 tuser=0
pacote=5 word=7 tsu=490 ns tdata=3220312e312f5054 tkeep=ff tvalid=1 tlast=0 tuser=0
pacote=5 word=8 tsu=500 ns tdata=430a0d4b4f203030 tkeep=ff tvalid=1 tlast=0 tuser=0
pacote=5 word=9 tsu=510 ns tdata=4c2d746e65746e6f tkeep=ff tvalid=1 tlast=0 tuser=0
pacote=5 word=10 tsu=520 ns tdata=31203a6874676e65 tkeep=ff tvalid=1 tlast=0 tuser=0
pacote=5 word=11 tsu=530 ns tdata=656e6e6f430a0d33 tkeep=ff tvalid=1 tlast=0 tuser=0
pacote=5 word=12 tsu=540 ns tdata=63203a6e6f697463 tkeep=ff tvalid=1 tlast=0 tuser=0
pacote=5 word=13 tsu=550 ns tdata=0a0d0a0d65736f6c tkeep=ff tvalid=1 tlast=0 tuser=0
pacote=5 word=14 tsu=560 ns tdata=57202c6f6c6c6548 tkeep=ff tvalid=1 tlast=0 tuser=0
pacote=5 word=15 tsu=570 ns tdata=00000021646c726f tkeep=1f tvalid=1 tlast=1 tuser=1
Pacote 5 concluido: bytes=125/125 words=16/16 timestamp=420/420 ns erro=0
pacote=6 word=0 tsu=590 ns tdata=0002012000000002 tkeep=ff tvalid=1 tlast=0 tuser=0
pacote=6 word=1 tsu=600 ns tdata=0045000801100000 tkeep=ff tvalid=1 tlast=0 tuser=0
pacote=6 word=2 tsu=610 ns tdata=1140000001003100 tkeep=ff tvalid=1 tlast=0 tuser=0
pacote=6 word=3 tsu=620 ns tdata=a8c00a0aa8c04ce5 tkeep=ff tvalid=1 tlast=0 tuser=0
pacote=6 word=4 tsu=630 ns tdata=1d00881308cf140a tkeep=ff tvalid=1 tlast=0 tuser=0
pacote=6 word=5 tsu=640 ns tdata=6761736e656d02ae tkeep=ff tvalid=1 tlast=0 tuser=0
pacote=6 word=6 tsu=650 ns tdata=6420504455206d65 tkeep=ff tvalid=1 tlast=0 tuser=0
pacote=6 word=7 tsu=660 ns tdata=0065747365742065 tkeep=7f tvalid=1 tlast=1 tuser=1
Pacote 6 concluido: bytes=63/63 words=8/8 timestamp=590/590 ns erro=0
pacote=7 word=0 tsu=680 ns tdata=0002011000000002 tkeep=ff tvalid=1 tlast=0 tuser=0
pacote=7 word=1 tsu=690 ns tdata=0045000801200000 tkeep=ff tvalid=1 tlast=0 tuser=0
pacote=7 word=2 tsu=700 ns tdata=1140000001003100 tkeep=ff tvalid=1 tlast=0 tuser=0
pacote=7 word=3 tsu=710 ns tdata=a8c0140aa8c04ce5 tkeep=ff tvalid=1 tlast=0 tuser=0
pacote=7 word=4 tsu=720 ns tdata=1d0008cf88130a0a tkeep=ff tvalid=1 tlast=0 tuser=0
pacote=7 word=5 tsu=730 ns tdata=736f707365720587 tkeep=ff tvalid=1 tlast=0 tuser=0
pacote=7 word=6 tsu=740 ns tdata=6420504455206174 tkeep=ff tvalid=1 tlast=0 tuser=0
pacote=7 word=7 tsu=750 ns tdata=0065747365742065 tkeep=7f tvalid=1 tlast=1 tuser=1
Pacote 7 concluido: bytes=63/63 words=8/8 timestamp=680/680 ns erro=0

```

=====

TESTE PASSOU: clock, reset, TKEEP, TLAST e TSU.

Pacotes processados: 7

Palavras processadas: 69

=====

O valor timestamp=10/10 ns significa que o timestamp produzido pelo DUT foi 10 ns e coincidiu com o valor esperado pelo testbench.

7 Resultado da simulacao realizada

Item	Resultado
Pacotes processados	7
Palavras processadas	69

Erros	0
Testes aprovados	clock, reset, TKEEP, TLAST e TSU

8 Como compilar e executar (FIZ NO COLAB)

```
!iverilog -g2012 -s tb_mac_mockup -o sim_mac mac_rx_mockup.sv  
tb_mac_mockup.sv  
!vvp sim_mac  
  
from google.colab import files  
files.download("resultado_simulacao.txt")
```